

2 Related Works

State Space Machines and Linear Transformers. There has been a recent surge of interest in state space machines and (kernelized) linear transformers (Gu et al., 2022; Katharopoulos et al., 2020; Peng et al., 2023; Sun et al., 2023; Gu & Dao, 2023; Fu et al., 2023; Poli et al., 2023; Luo et al., 2021; Peng et al., 2021; Wang et al., 2020), which are a class of models that combine the parallelizability of the Transformer with the memory efficiency of the RNN. These models can process both a sequential and a recurrent form, and can use the former for fast parallelizable training and the latter for memory-efficient inference. However, these models are still empirically inferior to the Transformer in terms of performance. Our work investigates the reasons behind this gap and proposes to close it by enhancing the in-context retrieval capability.

Chain of Thought (CoT). Chain of thought (Wei et al., 2023; Nye et al., 2021; Kojima et al., 2023; Wang & Zhou, 2024) is an augmentation to the Transformer, that allows it to solve more complex reasoning tasks by generating a reasoning process before outputting the answer. It has been shown that Transformers with CoT provably have more expressive power than the original Transformer without CoT (Feng et al., 2023; Li et al., 2024). However, the expressive power of RNNs with CoT has not yet been systematically studied. Theorem F.1 in Feng et al. (2023) shows that RNN cannot output a particular format of CoT for evaluating arithmetic expressions and solving linear equations while Transformers with the same amount of parameters can. Concurrent work (Yang et al., 2024) discovers that linear Transformers, a special class of RNNs, are not able to solve some dynamic programming problems with CoT, unless the number of parameters grows with the length of the input. One high-level message our work conveys is similar to theirs: RNNs have limited representation power to perform reasoning with CoT. However, we show that such limitation is not specific to the output format or architecture and apply tools from streaming complexity to prove lower bounds on a broader range of tasks and memory-efficient architectures.

Streaming Algorithms. Our lower bound leverages the technique in streaming algorithms. Streaming algorithms are algorithms that take constant (typically just 1) pass over the input and use sublinear space, hence including RNNs with fixed state space as a special case. Works in streaming algorithms date back to the 1980s (Munro & Paterson, 1980) and have been formalized and popularized in the 1990s (Alon et al., 1996) due to the need to process large data streams. The study of streaming algorithm is closely connected with the concept of **communication complexity**. The communication complexity of an algorithm is defined by the amount of communication cost required when the algorithm is distributed, and all streaming algorithms can be viewed as distributed algorithms with sublinear communication complexity, whose communication content is the internal state of the algorithm. The lower bound in our work is a direct application of this observation to the study of RNNs and we mainly consider the lower bounds for (1) indexing the input (Munro & Paterson, 1980) and (2) determining whether the input is a tree (Henzinger et al., 1998).

Retrieval Augmented Generation. Our work proposes to use retrieval augmentation to close the representation gap between RNNs and Transformers. This is consistent with the recent trend of retrieval augmented generation (Guu et al., 2020; Borgeaud et al., 2022; Rubin & Berant, 2023). Empirically, retrieval augmented generation has been shown to improve the performance of recurrent models in various tasks (Kuratov et al., 2024; Akyürek et al., 2024) and our work provides a theoretical foundation for this phenomenon. Our work also shows that an attention layer can be used to simulate the retrieval process, which is consistent with the finding that attention can improve the performance of RNNs (Vaswani et al., 2017; Arora et al., 2023; Park et al., 2024; Peng et al., 2023; Hao et al., 2019). It has also been shown empirically that attention can be used to simulate complex retrieval process (Jiang et al., 2022).

Comparison Between Transformers and RNNs (Without CoT). A line of works focused on the comparison between RNNs and Transformers in terms of recognizing or generating formal languages (Bhattamishra et al., 2020; Hahn, 2020; Merrill et al., 2022). These works show that the lack of recurrent structure in Transformers makes them fail to recognize some formal languages that RNNs can recognize. However, Liu et al. (2023); Yao et al. (2023); Hao et al. (2022) show that such limitation can be mitigated when we consider bounded length of input or bounded grammar depth. Our work differs from these works in that we consider the expressive power of RNNs and Transformers with CoT and show that in this case, the gap between RNNs and Transformers is one-sided (Theorem 4.8).

Prior work (Arora et al., 2023) has shown that input-independent gating SSMs are inferior to Transformers in the task called *associative recall* (Willshaw et al., 1969; Hopfield, 1982; Hinton & Anderson, 2014). The task requires the model to recall a previously seen pattern given a partial input. They show that input-dependent gating SSMs have better performance in associative recall and also propose a hybrid architecture that combines input-independent state space machines with attention to achieve better performance. Our work differs from this work in the following ways: (1) Our work studies associative recall from a theoretical perspective and proves formal lower bounds on the memory size of RNNs necessary for solving associative recall and other retrieval tasks; (2) We also study hybrid architectures but we provide a proof that appending a single Transformer layer to RNNs can make them expressive enough; (3) Our theory applies to not only input-independent gating SSMs but also all RNNs with $o(n)$ -bit memory.

Prior work (Jelassi et al., 2024) proves a representation gap between RNNs and Transformers in repeating a long sequence, which can be seen as a retrieval task. They show that RNNs have difficulty performing the task due to their limited memory. Our work further proves that RNNs are limited in solving many other retrieval tasks, even with CoT. Technically, a key ingredient in their proof is a counting argument on the output sequence to show a limited memory size is not enough to produce too many different output sequences, but our proof can handle retrieval tasks that only require outputting a single token.

Notably, Sanford et al. (2023) apply communication complexity to prove circuit size or memory size lower bounds for RNNs and Transformers on the task of sparse averaging. Sanford et al. (2024) extend this technique to another task called hop_k , a generalization of the associative recall task. Our technique is similar to theirs since our proof is also based on communication complexity. But we consider a broader range of tasks including seemingly irrelevant reasoning tasks such as IsTree, and further explore various ways to close the representation gap.

Representation Theory of RNNs. Another line of works (Li et al., 2021, 2022; Alberti et al., 2023) studies the universal approximation power of RNNs. They show that the upper bound of the approximation power of linear RNNs will be constrained by the dimension of the hidden states. Their works on the high level are consistent with our findings but are not directly comparable because we are considering finite precision compute models with the assistance of CoT or In-context RAG.

3 Preliminaries

We introduce the definitions that are necessary for understanding our results and defer other definitions to Appendix A.

Vocabulary and Embeddings. A vocabulary V is a finite set of tokens. A word embedding for a token $v \in V$ is a vector $W_v^{(E)} \in \mathbb{R}^d$ that represents the token, and a position embedding for the k -th token in a sequence is a vector $W_k^{(P)} \in \mathbb{R}^d$ that represents the position. Given a sequence of tokens $\mathcal{S}$, an embedding function $\text{Emb}(\mathcal{S})$ maps each token to a vector in $\mathbb{R}^d$ by mixing word and position embeddings, resulting in a sequence of vectors. To ease our comparison between RNNs and Transformers, in this paper, we assume fixed word and position embeddings and do not learn them during training. See Appendix A.3 for the formal definitions. This is common practice and has been used in many previous works studying the theory of Transformers (Li et al., 2023b; Tian et al., 2023).

Many of the problems we study involve natural numbers up to n , where the input sequence length is linear in n . For simplicity, we assume the vocabulary contains $[n] = \{1, \dots, n\}$ and the word embedding for i is defined as $W_i^{(E)} = iw_1$, where w_1 is the first coordinate vector. But in practice, the vocabulary size does not increase with n and numbers may be tokenized into a few tokens according to their decimal representations. We note that our results can be easily extended to this more practical case since our lower bounds do not rely on the specific form of the vocabulary and embeddings and for the upper bounds, our embedding can be easily represented by a few RNN or Transformer layers.

Numerical Precision. We will consider computation models with fixed numerical precision in this paper. We will use p to denote the precision of the number of bits to represent real numbers and use $\mathbb{R}_p$ to denote the set of all real numbers that can be represented by p -bit floating point numbers. We defer the details to Appendix A.2. We will assume $p = O(\log n)$ in this paper and state the constant explicitly when necessary. This is a common assumption when studying the finite precision neural networks (Feng et al., 2023; Merrill & Sabharwal, 2023).

Language Modeling. We use V^* and V^+ to denote the set of all finite sequences and all non-empty finite sequences of tokens in V , respectively. We study language models that can predict the next token given a prefix of tokens. For this, we define a language model (LM) as a function $M : V^+ \rightarrow \mathbb{P}_V$ that maps a non-empty sequence to a probability distribution over the next token, where $\mathbb{P}_V$ is the probability simplex over V . We specifically study the case where the language model is realized by deep neural networks: first map the input sequence $\mathcal{S}$ into a sequence of embeddings $\text{Emb}(\mathcal{S})$, and then apply a neural network, such as a Transformer or RNN, to process the embeddings and output the probability distribution. We would call a series of parameterized models with increasing input size a *family* of models.

Transformer. We will first define the Transformer architecture used in the theoretical analysis in this paper.

Definition 3.1 (Transformer Block). Let $X \in \mathbb{R}^{d \times l}$ be the input matrix, where l is the sequence length. The output of a Transformer block f is defined as:

$$f(X) = X + \mathcal{A}(X) + g(X + \mathcal{A}(X)),$$

$$\mathcal{A}(X) = \sum_{h=1}^H W^{(V,h)} X \text{softmax} \left(\frac{(W^{(K,h)} X)^\top W^{(Q,h)} X}{\sqrt{d}} + C \right), \quad (1)$$

where g is a column-wise ReGLU feed-forward network⁴ with width w and output dimension d , $\mathcal{A}$ is the scaled dot-product attention, softmax is the column-wise softmax function, $W^{(K,h)}$, $W^{(Q,h)}$, $W^{(V,h)}$ are the learnable

parameters and H is the number of heads, and $C = \begin{bmatrix} 0 & 0 & \dots & 0 \\ -\infty & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ -\infty & -\infty & \dots & 0 \end{bmatrix} \in \mathbb{R}^{l \times l}$ is a mask to prevent the attention

from attending to future tokens.

In the context of language modeling, given a sequence of tokens $\mathcal{S}$, a Transformer $T(\mathcal{S})$ is defined as:

Definition 3.2 (Transformer). Let $\mathcal{S} \in |V|^l$ be the tokenized input sequence, the output of a Transformer is defined as:

$$T(\mathcal{S}) = \text{softmax} \left(W^{(E)} (f_L (\dots f_1 (\text{Emb}(\mathcal{S})))) \right)_{:,l}. \quad (2)$$

where softmax is the column-wise softmax function, f_i is the i -th Transformer block. We will call the i -th Transformer block the i -th layer of the Transformer and denote its feed-forward layer and attention layer as g_i and $\mathcal{A}_i$ respectively.

Recurrent Neural Networks Recently there has been a lot of interest in the linear-time Transformer, which replaces the full-attention calculation with linear-time alternatives. These variants are mostly special forms of recurrent neural networks (RNNs) that are parallelizable. Here we define a general form of RNNs containing all the common variants to the best of our knowledge, including Mamba (Gu & Dao, 2023), RWKV (Peng et al., 2023), RetNet (Sun et al., 2023), StreamingLLM (Xiao et al., 2023), RMT (Bulatov et al., 2022), TOVA (Oren et al., 2024), xLSTM (Beck et al., 2024) etc. An RNN maintains a state $s_t \in \mathbb{R}_p^\Lambda$ storing Λ p -bit floating point numbers.

Definition 3.3 (RNN). An RNN architecture is characterized by two functions: state transition function $\mathbf{t} : \Theta \rightarrow (\mathbb{R}_p^\Lambda \times \mathbb{R}_p^d \rightarrow \mathbb{R}_p^\Lambda)$ and output function $\mathbf{o} : \Theta \rightarrow (\mathbb{R}_p^\Lambda \rightarrow \mathbb{R}_p^d)$, where Λ is the dimension of the state and Θ is the parameter space. Let $\mathcal{S} \in |V|^l$ be the input sequence, the output of a recurrent neural network with parameter $\theta \in \Theta$ is defined as:

$$R_\theta(\mathcal{S}) = \text{softmax} \left(W^{(E)} \mathbf{o}_\theta(s_l) \right), \\ \forall k \in [l], s_k = \mathbf{t}_\theta(s_{k-1}, \text{Emb}(\mathcal{S})_{:,k}),$$

where $s_0 \in \mathbb{R}_p^\Lambda$ is a vector determined by θ and $W^{(E)}$ is the word embedding matrix. We will omit the subscript θ when it is clear from the context.

We can characterize the complexity of an RNN architecture with the following three measures,

1. Parameter size P: the number of bit of parameters determining $\mathbf{t}$ and $\mathbf{o}$.
2. State memory size M: the number of bits to record the state of the RNN, in this case, is $\Lambda \times p$.
3. Circuit size C: the number of bit-wise arithmetic operations needed to calculate $\mathbf{t}$ and $\mathbf{o}$.

A particularly interesting class of RNNs is constant-size RNNs, where the dimension of state and number of parameters are fixed and do not depend on n , i.e., $\Lambda = O(1)$, $p = O(\log n)$, $P = O(\log n)$, $M = O(\log n)$, $C = O(\log n)$.

We do not assume a specific structure of the RNN when we want to prove impossibility results for RNNs, i.e., what RNNs cannot represent. But when we want to showcase what RNNs can represent, we focus on Linear RNNs, one of the simplest form of RNNs, so that our results are more likely to be generalizable to more complex RNNs.

Definition 3.4 (RNN block). A Linear RNN block is defined as follows:

$$f(X) = X + \text{LU}(X) + g(X + \text{LU}(X)),$$

where g is a column-wise ReGLU feed-forward network with width w and output dimension d and LU is a linear unit, defined as

$$h_0 = 0, h_{:,t} = A h_{:,t-1} + B X_{:,t}, \text{LU}(X_{:,1:l}) = h_{:,1:l}.$$

Definition 3.5 (Linear RNN). A Linear RNN is a recurrent neural network

$$R(\mathcal{S}) = \text{softmax} \left(W^{(E)} (f_L (\dots f_1 (\text{Emb}(\mathcal{S})))) \right)_{:,l}. \quad (3)$$

where softmax is the column-wise softmax function, f_i is the i -th Linear RNN block. We will call the i -th Linear RNN block the i -th layer of the Linear RNN and denote its feed-forward layer and linear unit layer as g_i and LU_i respectively.

⁴ReGLU means $\sigma(x) = \text{ReLU}(W_1 x + b_1) \otimes (W_2 x + b_2)$, this is a surrogate for the commonly used SwiGLU activation and allows the model to perform multiplication of two coordinates.